Episodio 1 de 14: Qué hicimos y qué salió

Serie «Leer sin que te quiten el libro»: la réplica del paper «A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting» (Euro-Par 2024), explicada para principiantes.

- **Episodio anterior:** es el primero de la serie.
- **Episodio siguiente:** episodio 2, «La carrera entre leer y liberar».

Lo esencial de este episodio:

- El equipo repitió el paper con los programas oficiales de los autores en un PC con un Intel Core Ultra 9 285, con y sin Docker: más de 14 000 mediciones, ninguna fallida.
- De las 15 afirmaciones del paper, 9 se reproducen, 2 en parte, 3 no y 1 no se confirma.
- Lo central sale bien parado: es correcto, cumple su cota wait-free, no acumula basura y lee rápido. No se sostiene la superioridad frente a Folly ni la ventaja con un millón de pilas.

Parte 0. El resumen en dos minutos

Qué se hizo. Se repitió el paper en un PC con un Intel Core Ultra 9 285 de 24 núcleos, con sus cinco algoritmos y sus programas oficiales. Este PC no tiene 4 chips, así que esa distancia se simuló colocando los hilos en núcleos concretos. Después se repitió todo sin Docker, con los mismos programas oficiales y con el programa del equipo, y se añadieron controles para entender cada diferencia.

Tanda	Algoritmos	Mediciones	Fallidas
Con Docker, programas oficiales	Autores, Anderson, Folly, Williams, Libstdc++	3 720	0
Sin Docker, programas oficiales	Los mismos 5	4 660	0
Sin Docker, `bench.cpp` del equipo	Autores, Williams, Libstdc++, Mutex	2 976	0
Sin Docker, `bench.cpp` con las pilas alineadas	Los mismos 4	2 160	0
Controles	Según el caso	546	0
Tráfico real de RAM	Los 5 del paper y el techo de la máquina	70	0
Detectores de errores en los rivales	Los 5 del paper y Mutex	26	0

Las respuestas cortas

Pregunta	Respuesta
¿El algoritmo del paper es correcto?	Sí en lo probado: 0 errores con ASan y TSan
¿Cumple su promesa wait-free?	Sí: reintentó como máximo 8 veces, con una cota de 32

Pregunta	Respuesta
¿Deja memoria sin liberar?	No. Anderson sí retiene algo: 469 KB con 24 hilos
¿Tiene el `load` más rápido?	Sí, aunque con contención por poco
¿Gana a Libstdc++ y a Williams?	Sí, con y sin Docker
¿Gana a Folly?	Aquí no: gana 3 escenarios, empata 2 y pierde 1, con y sin Docker. La causa sigue sin aclarar
¿Gana a Anderson?	Depende. Con pocas escrituras, sí. Con muchas sobre pocas pilas, Anderson llega a 1,99×
¿Importa dónde cae cada hilo?	Con contención, mucho: hasta −55 %. Sin contención, casi nada: ±3 % como mucho
¿Docker cambia los resultados?	No: los programas oficiales rinden lo mismo dentro y fuera (mediana de 0,99× a 1,01× según el algoritmo). La excepción es Williams entre clústeres, y ahí cambian más cosas que Docker
¿Los rivales rompen la memoria?	No en lo probado: 0 avisos en los seis
¿Por qué todos empatan con un millón de pilas?	Muy probablemente por la RAM: Anderson, Libstdc++ y Folly llegan al 89-93 % de lo que da este PC, que tiene un solo módulo de memoria

Lo más valioso que salió por el camino

1. **`bench.cpp` del equipo tiene compartición falsa.** Sus 10 pilas caben en 2 líneas de caché, y eso frena a Autores hasta 3,65 veces. Con una sola palabra añadida, sube de 32 a 119 Mop/s, casi los 122 del programa oficial. La batería completa con esa palabra lo confirma: de 35 a 120 Mop/s. Ver la Parte 6.6.

2. **El binario oficial de latencia se compila sin `-mcx16`.** Con eso, Williams lleva un candado escondido. Lo medimos mal al principio; ya está corregido, y cambia el veredicto: con contención **no** es el peor. Ver la Parte 6.2.

3. **Importa compartir caché, no tanto la distancia.** Dos hilos que comparten L2 rinden casi el doble que cualquier otra pareja, y da igual que el viaje dure 47 o 86 ns. Ver la Parte 6.4.

4. **Williams se hunde entre clústeres, pero solo en el programa oficial.** Con Docker cae un 97 % en los 7 pares probados. Sin Docker también se hunde, aunque menos: un 76 %, con rondas que saltan de 0,34 a 1,79 Mop/s. Así que Docker no lo provoca, pero algo de lo que cambia entre las dos tandas cambia cuánto. Se descartaron también `boost::atomic` y un par defectuoso. La causa sigue abierta. Ver la Parte 6.4.

5. **Con un millón de pilas, el freno es sobre todo la RAM.** Los contadores de la CPU muestran que este PC mueve unos 37 GB/s en accesos al azar, y Anderson, Libstdc++ y Folly llegan a 33-34. Ver la Parte 6.1.

6. **Docker no cambia los programas oficiales.** Se midieron dentro y fuera: de los 270 puntos de la figura 5, 259 cambian menos de un 10 %. Antes era una deducción; ahora está medido. Ver la Parte 6.6.

Lo que falta. Ya no queda nada por medir de lo planeado: la tanda sin Docker con los programas oficiales terminó el 7 de noviembre de 2026, sin un solo fallo. Lo que queda son

preguntas: por qué Folly rinde aquí mucho más que en el paper, por qué Williams se hunde entre clústeres, por qué Autores va por detrás con n = 10^6 y pocos hilos, y si la RAM es de verdad el techo. Además, la latencia con contención sigue cambiando mucho de una ronda a otra, incluso con 15 rondas, así que sus diferencias pequeñas no valen como victoria. La Parte 8 lo cuenta y la Parte 9 explica cómo repetirlo.
